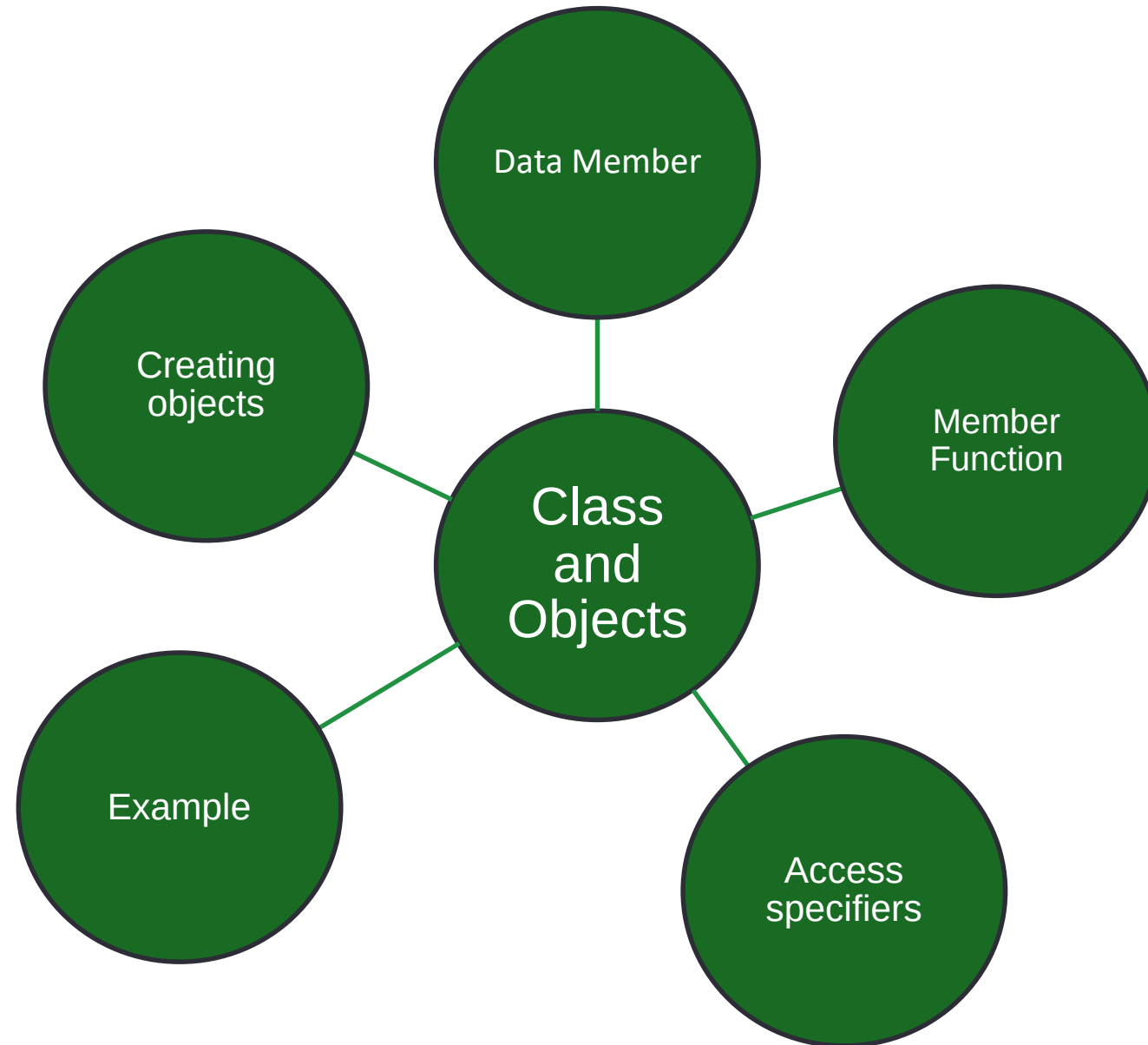


Class and Objects



Class Introduction

- ▶ The class declaration describes the type and scope of its members.
- ▶ The class function definitions describe how the class functions are implemented.
- ▶ The class body contains the declarations of variables and functions. These variables and functions are collectively called members. The variables are called data members or member variables and the functions are called member functions.

Class Introduction

- ▶ They are usually grouped under two sections, private and public.
- ▶ The keyword private and public are called visibility labels.
- ▶ The private members can be accessed only from within the class and the public members can be accessed from outside the class.
- ▶ The use of keyword private is optional. By default all the members of class are private.
- ▶ Only the member functions can access the private members of class.

Specifying a class

- ▶ The class is a way of binding data and functions associated with data together.
- ▶ The entire set of data and the functions can be made user defined data type with help of class. Thus Class is nothing but a user defined data type.
- ▶ Class act as a template, which is used to create objects. The class contains data and member functions that operate on the data.
- ▶ It allows the data to be hidden, if necessary from the external use.
- ▶ Generally class specification has two parts
 - ❖ class declaration
 - ❖ class function definitions.

Syntax of class



```
class class-name
{
    private :
        variable declarations;
        function declarations;
    public:
        variable declarations;
        function declarations;
};
```

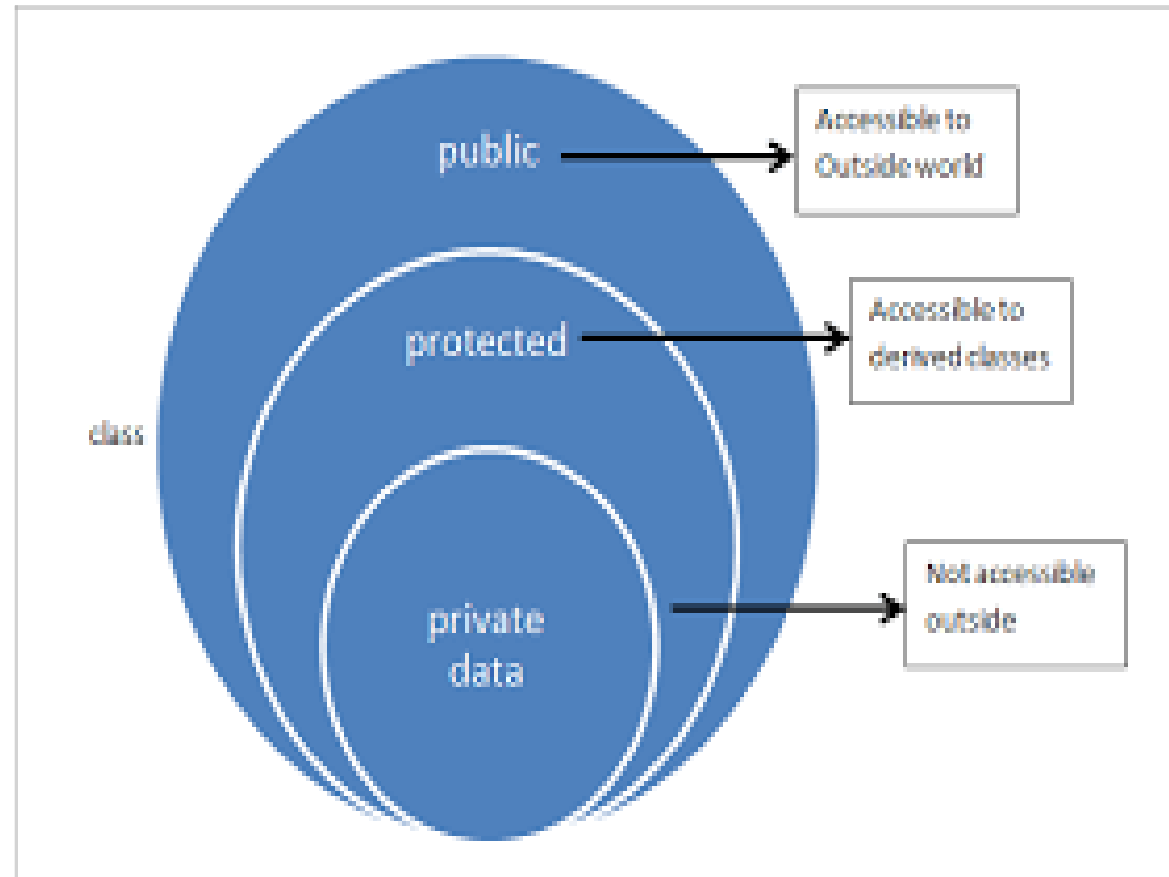
Example



Class student

```
{  
private :  
    int rollno;           // data member  
    char name[30];       // data member  
public:  
    void getdata (void);  // member function  
    void showdata(void);  // member function  
};
```

ACCESS SPECIFIERS



Concept Explanation: Access Specifiers in C++

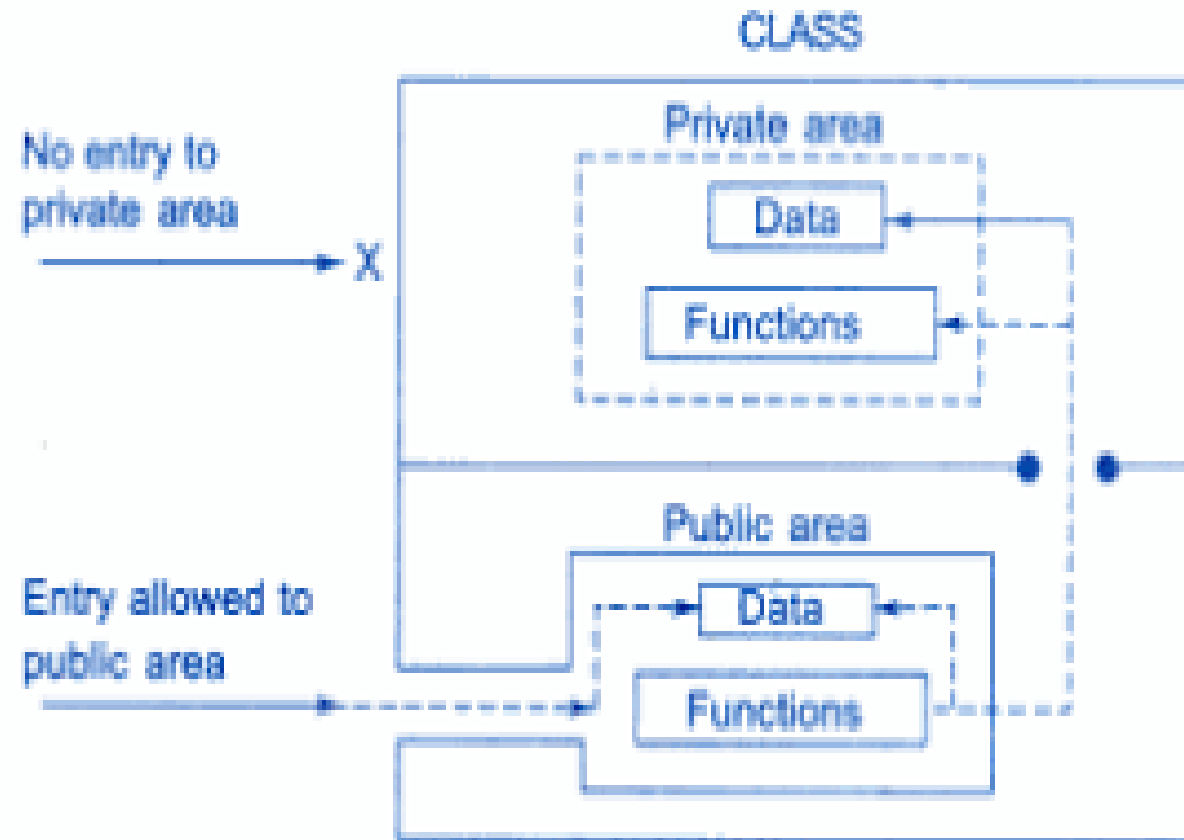
- ▶ In C++ Access specifiers are also called as visibility mode. They are as follows: **Public**, **Private**, and **Protected**.
- ▶ **Public**:- When access specifier is public i.e. base class is publicly inherited by derived class all public member of base class become public members of derived class and all protected members become protected in derived class.
- ▶ **Private**:- All public and protected members of base class become private members of derived class.
- ▶ **Protected**:- In this all public and protected members of base class become protected of derived class

- Private – No Direct Access
- Protected – No Direct Access
- Public – Direct Access

Access-control specifiers	Accessible to	
	Own class members	Objects of a class
Private	Yes	No
Protected	Yes	No
Public	Yes	Yes

- ▶ OOP emphasis on data rather than procedures.
- ▶ The primary focus is on Data.
- ▶ In OOP the data and the functions that operate on the data are encapsulated in to a single unit known as object.
- ▶ Thus object contains data and the member functions that operate on the data.
- ▶ Only the member functions can access the private data of the class. The private data is not accessible to the external functions. This insulation of data from the external functions is called Data Hiding or Information Hiding

Data Hiding concept



Example



```
class MyClass {  
    public: // Public access specifier  
    int x; // Public attribute  
    private: // Private access specifier  
    int y; // Private attribute  
};  
  
int main() {  
    MyClass myObj;  
    myObj.x = 25; // Allowed (public)  
    myObj.y = 50; // Not allowed (private)  
    return 0;  
}
```

Example

Note: It is possible to access private members of a class using a public method inside the same class.

Tip: It is considered good practice to declare your class attributes as private (as often as you can). This will reduce the possibility of yourself (or others) to mess up the code. Note: By default, all members of a class are private if you don't specify an access specifier:

Example

```
class MyClass {  
    int x; // Private attribute  
    int y; // Private attribute  
};
```

Concept Explanation: Method of defining member function

- The **functions**, which are declared inside the class, are called **member functions** of that class.
- Only member functions can access the private members of the class.
- Generally member functions are placed in a public section. But they can be placed in a private section also.
- **Member function** can be defined in **two places**.
 - 1) **Outside** the class definition
 - 2) **Inside** the class definition

Syntax of inside member function declaration in a class

► Syntax:

```
class class-name
```

```
{
```

```
public:
```

```
Return_type member function name(parameters list or argument)
```

```
{
```

```
Statements;
```

```
}
```

```
};
```

- where class is keyword and class name is name of class want to declare
- Return type is return type of member function that can be void, int ,char, float
- Member function is the name of function that want to declare
- Parameter list or arguments want to give to your function.
- Argument syntax can be

(Data_type1 variable name1, Data_type2 variable name2,....., Data_typen variable namen)

Example of inside member function declaration in a class

- Write a program to declare a class 'emp' containing data member's emp_id and salary. Accept and display this data for 1 object of the class.

```
#include<iostream>
using namespace std;
class Emp
{
private:
    int emp_id;
    float salary;

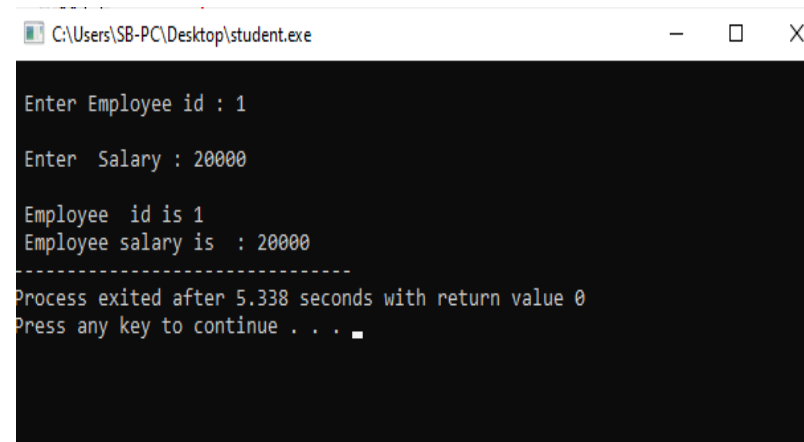
public:
    void accept();
    void showdata();
};

void Emp::accept()
{
    cout<<"\n Enter Employee id : ";
    cin>>emp_id;
    cout<<"\n Enter Salary : ";
    cin>>salary;
}
```

```
void Emp::showdata()
{
    cout<<"\n Employee id is "<<emp_id;
    cout<<"\n Employee salary is : "<<salary;
}

int main()
{
    Emp e;
    e.accept();
    e.showdata();
}
```

Output:
Executed with the help of Dev C++



Syntax of Outside member function declaration in a class

- ▶ In this, the member functions that are declared inside the class have to be defined outside the class.
- ▶ The member function incorporates a 'membership identity label' in the header.
- ▶ This label tells the compiler which class the function belongs to.
- ▶ The general syntax of defining any member function outside the class is as follows

- ▶ Syntax:

```
class class-name
{
public:
Return_type member function name(parameters list or argument);
};
Return_type class_name:: member function name(parameters list or argument);
{
Statements;
}
```

- ▶ The membership label 'class-name:: ' tells the compiler that the function belongs to the specified class.
- ▶ The symbol "::" is called scope resolution operator.

Example of outside member function declaration

Class student

```
{
    private :int roll-no;
            float marks;
    public :void  getdata (void);           // member function declaration
            void  showdata (void); // member function declaration
};

void student :: getdata (void)             // member function definition outside the class
{
    cout<< "Enter Roll No : ";
    cin >> roll-no;
    cout<< "Enter Marks : ";
    cin >> marks;
}

void student :: showdata (void)           // member function definition outside the class
{ cout<<" Roll No = " << roll-no << endl;
  cout<<" Marks = " << marks << endl;
```

Example of inside member function declaration in a class

- Write a program to declare a class 'emp' containing data member's emp_id and salary. Accept and display this data for 1 object of the class.

```
#include<iostream>
using namespace std;
class Emp
{
private:
    int emp_id;
    float salary;

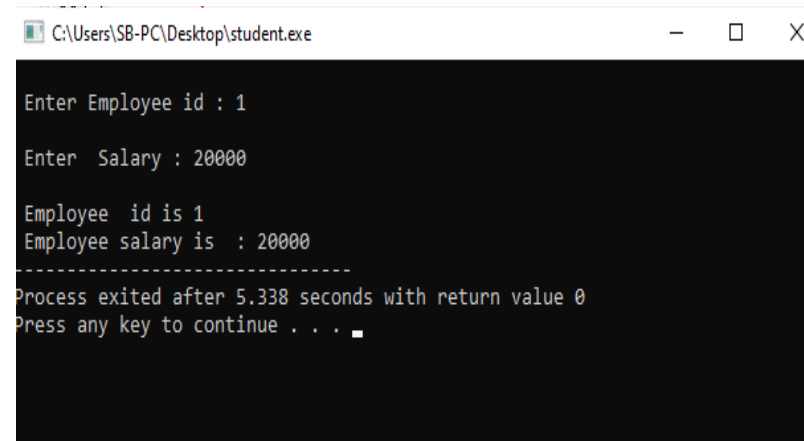
public:
    void accept();
    void showdata();
};

void Emp::accept()
{
    cout<<"\n Enter Employee id : ";
    cin>>emp_id;
    cout<<"\n Enter Salary : ";
    cin>>salary;
}
```

```
void Emp::showdata()
{
    cout<<"\n Employee id is "<<emp_id;
    cout<<"\n Employee salary is : "<<salary;
}

int main()
{
    Emp e;
    e.accept();
    e.showdata();
}
```

Output:
Executed with the help of Dev C++

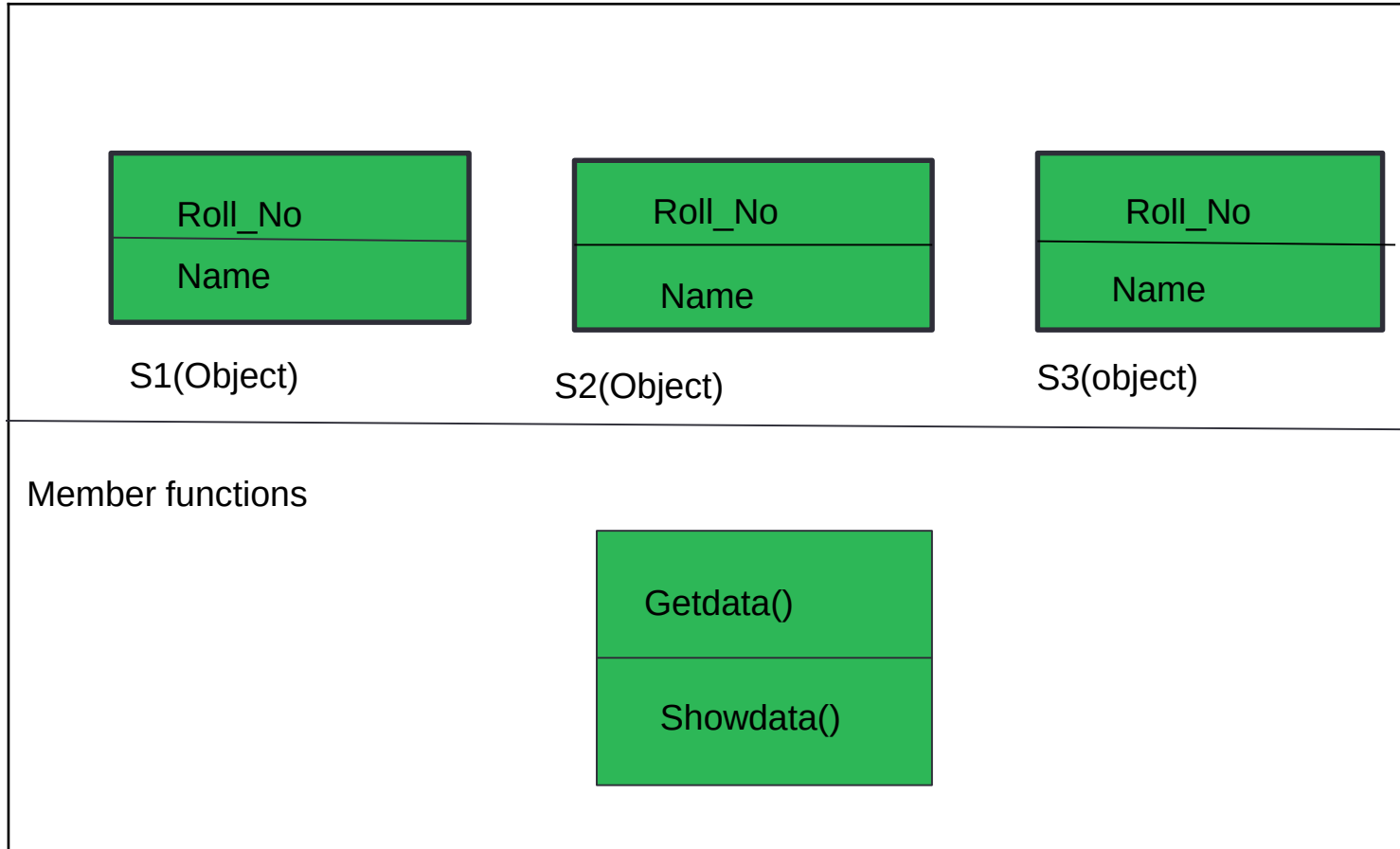


```
C:\Users\SB-PC\Desktop\student.exe
Enter Employee id : 1
Enter Salary : 20000

Employee id is 1
Employee salary is : 20000
-----
Process exited after 5.338 seconds with return value 0
Press any key to continue . . .
```

- ▶ Memory space for object is allocated when they are declared and not when the class is specified.
- ▶ But the member functions are created and placed in the memory space only once when they are defined as a part of the class specification.
- ▶ As all objects belongs to same class, use the same member functions, no separate memory is allocated for member functions when the objects are created i.e. when the objects are created the memory space is allocated only for data members (member variables)
- ▶ Space for data members is allocated separately for each object.
- ▶ The memory locations are also distinct for object of same class because the data members of different object will hold different values.
- ▶ Data member of the class can contain different value for the different object, memory allocation is performed separately for each data member for different object at the time of creating an object.
- ▶ Member function remains common for all object. Memory allocation is done only once for member function at the time of defining it.

Example



- Consider class student having two data member and three member functions.
- Student s1,s2,s3; three objects for class student.
- Data members Roll_No and name required separate memory space as per object.
- Object s1 requires separate memory as so on
- Member functions same for all objects. getdata and showdata requires same memory space

Example:

